

# Energy Managment System - EMS Energy

Michał Jędrzejczyk, Dawid Frontczak

|                  |                                         |
|------------------|-----------------------------------------|
| Środowisko       | Ignition 8.3, PostgreSQL, Perspective   |
| Moduł            | Energy Management System                |
| Baza danych      | ems_energy                              |
| Typ danych       | pomiary energii, mocy, napięcia i prądu |
| Data opracowania | czerwiec 2026                           |

*Dokumentacja techniczna systemu EMS Energy przygotowanego w Ignition Perspective.*

## 1. Cel i zakres projektu

Celem projektu było przygotowanie działającego systemu Energy Management System (EMS) w środowisku Ignition 8.3. System został wykonany dla modułu Energy i umożliwia monitorowanie danych energetycznych, prezentowanie ich w dashboardzie Perspective, obsługę alarmów oraz integrację z bazą PostgreSQL.

Projekt obejmuje konfigurację bazy danych, połączenia z Ignition Gateway, symulatora urządzenia, struktury tagów, skryptów backendowych oraz interfejsu użytkownika. Dashboard pozwala analizować dane według lokalizacji, licznika, zakresu czasu oraz limitu liczby rekordów.

- bieżący podgląd KPI: moc, energia, napięcie, współczynnik/moc oraz liczba alarmów;
- tabele analityczne dla trendu energii, zużycia według lokalizacji i przeglądu liczników;
- komponent Alarm Status Table prezentujący aktywne alarmy;
- wykresy czasowe prezentujące trendy danych pomiarowych;
- skrypty Project Library odpowiedzialne za logikę KPI, tabel, alarmów i stylów.

## 2. Architektura systemu

System został przygotowany w architekturze warstwowej. Warstwa danych opiera się na PostgreSQL, warstwa komunikacji i archiwizacji działa w Ignition Gateway, natomiast warstwa prezentacji została wykonana w module Perspective. Dane wejściowe są dostarczane przez symulator urządzenia oraz tagi w domyślnym providerze.

| Warstwa          | Opis                                                                                 |
|------------------|--------------------------------------------------------------------------------------|
| PostgreSQL       | Baza ems_energy przechowuje strukturę systemu, pomiary, alarmy, użytkowników i role. |
| Ignition Gateway | Obsługuje połączenia, historian, device connection i komunikację z bazą.             |
| Tag Provider     | Zawiera strukturę EMS/Energy z licznikami i wartościami pomiarowymi.                 |

## Perspective Dashboard

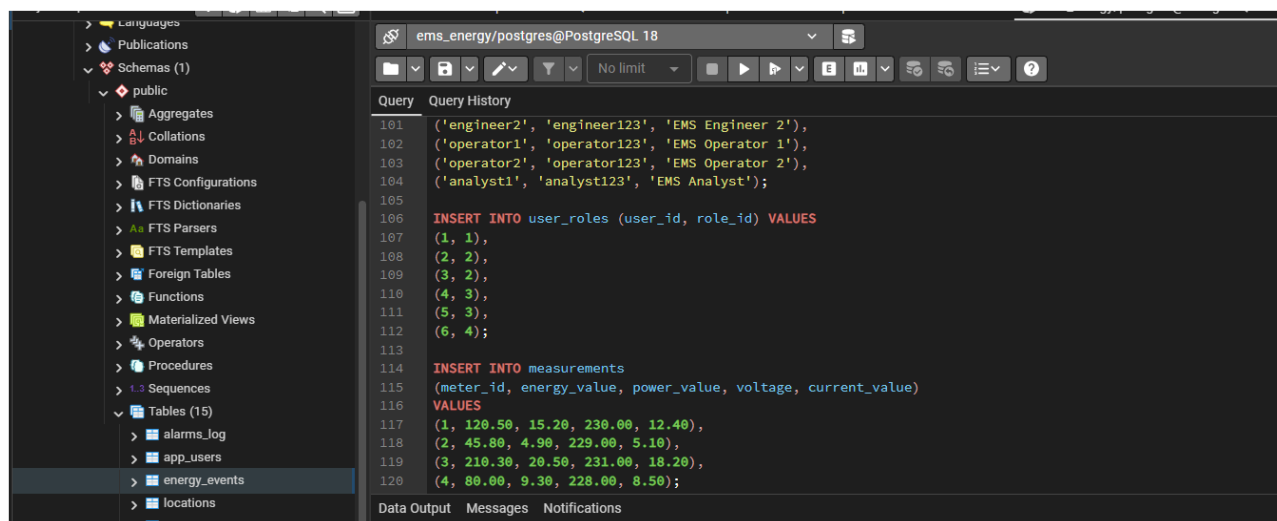
Prezentuje KPI, tabele, wykresy, alarmy i filtry użytkownika.

## 2.1. Przepływ danych

- symulator enea generuje lub udostępnia dane pomiarowe;
- tagi w Ignition przechowują wartości Current, EnergyValue, PowerValue i Voltage;
- skrypty Project Library pobierają dane z tagów i przeliczają je na potrzeby KPI oraz tabel;
- dashboard Perspective dynamicznie prezentuje wyniki oraz alarmy;
- PostgreSQL oraz historian zapewniają warstwę trwałego zapisu i archiwizacji.

## 3. Baza danych PostgreSQL

Baza danych PostgreSQL została utworzona pod nazwą `ems_energy`. Jej struktura zawiera tabele konfiguracyjne, pomiarowe, alarmowe oraz tabele użytkowników i ról. Taka organizacja umożliwia rozdzielenie danych technicznych od danych związanych z bezpieczeństwem oraz zdarzeniami systemowymi.



Rys. 1. Struktura tabel bazy danych PostgreSQL `ems_energy`.

W bazie znajdują się m.in. tabele `locations`, `meters`, `measurements`, `alarms_log`, `energy_events`, `app_users`, `roles` oraz `user_roles`. Tabela `measurements` przechowuje przykładowe wartości energii, mocy, napięcia i prądu przypisane do konkretnego licznika.

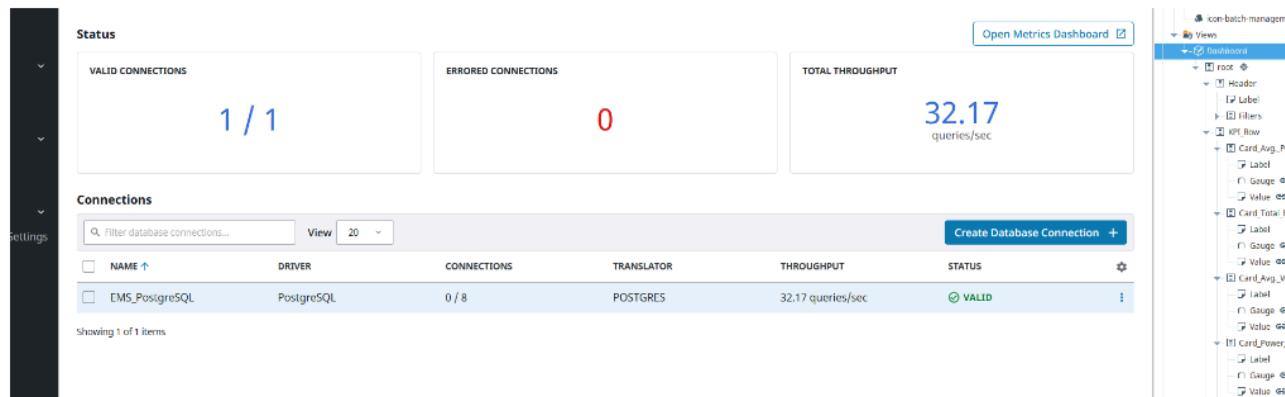
### 3.1. Najważniejsze tabele

| Tabela                                                                | Przeznaczenie                                   |
|-----------------------------------------------------------------------|-------------------------------------------------|
| <code>locations</code>                                                | lista lokalizacji i obszarów pomiarowych        |
| <code>meters</code>                                                   | definicje liczników przypisanych do lokalizacji |
| <code>measurements</code>                                             | dane pomiarowe energii, mocy, napięcia i prądu  |
| <code>alarms_log</code>                                               | rejestr alarmów i zdarzeń alarmowych            |
| <code>energy_events</code>                                            | zdarzenia związane z pracą systemu Energy       |
| <code>app_users</code> , <code>roles</code> , <code>user_roles</code> | struktura użytkowników i ról bezpieczeństwa     |

## 4. Konfiguracja Ignition Gateway

### 4.1. Połączenie z bazą danych

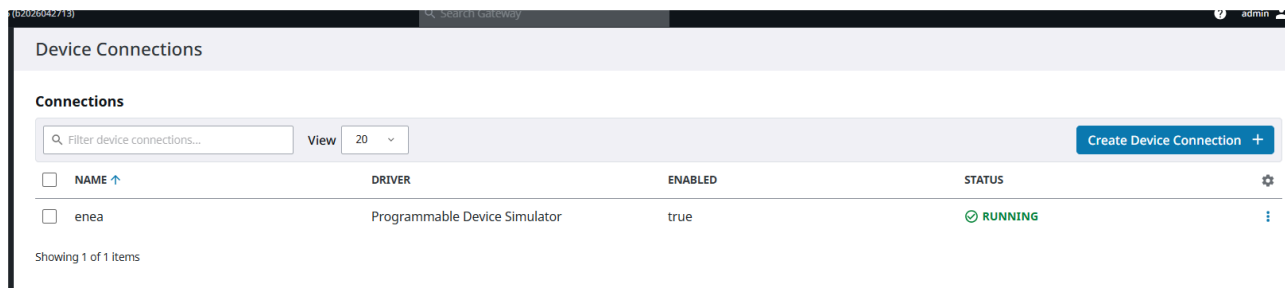
Połączenie z bazą PostgreSQL zostało skonfigurowane w Ignition Gateway jako EMS\_PostgreSQL. Status VALID potwierdza poprawną komunikację między Ignition a bazą danych. Dzięki temu projekt może wykonywać zapytania SQL oraz korzystać z danych pomiarowych i konfiguracyjnych.



Rys. 2. Połączenie EMS\_PostgreSQL ze statusem VALID.

### 4.2. Symulator urządzenia

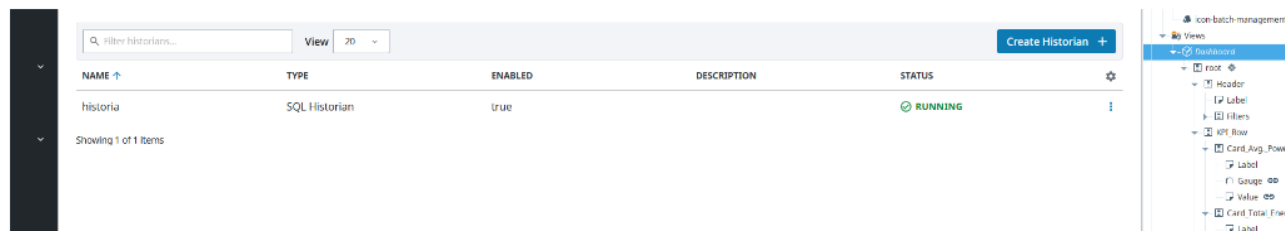
Źródłem danych dla projektu jest urządzenie enea skonfigurowane jako Programmable Device Simulator. Status RUNNING oznacza, że symulator działa i może zasilać system wartościami pomiarowymi wykorzystywanymi przez tagi, dashboard i alarmy.



Rys. 3. Device Connection enea ze statusem RUNNING.

### 4.3. SQL Historian

W projekcie skonfigurowano SQL Historian o nazwie historia. Historian jest włączony i ma status RUNNING, co umożliwia archiwizację danych tagów oraz późniejsze wykorzystanie ich w trendach i analizach czasowych.



Rys. 4. SQL Historian historia działający w Ignition Gateway.

## 5. Struktura tagów i alarmów

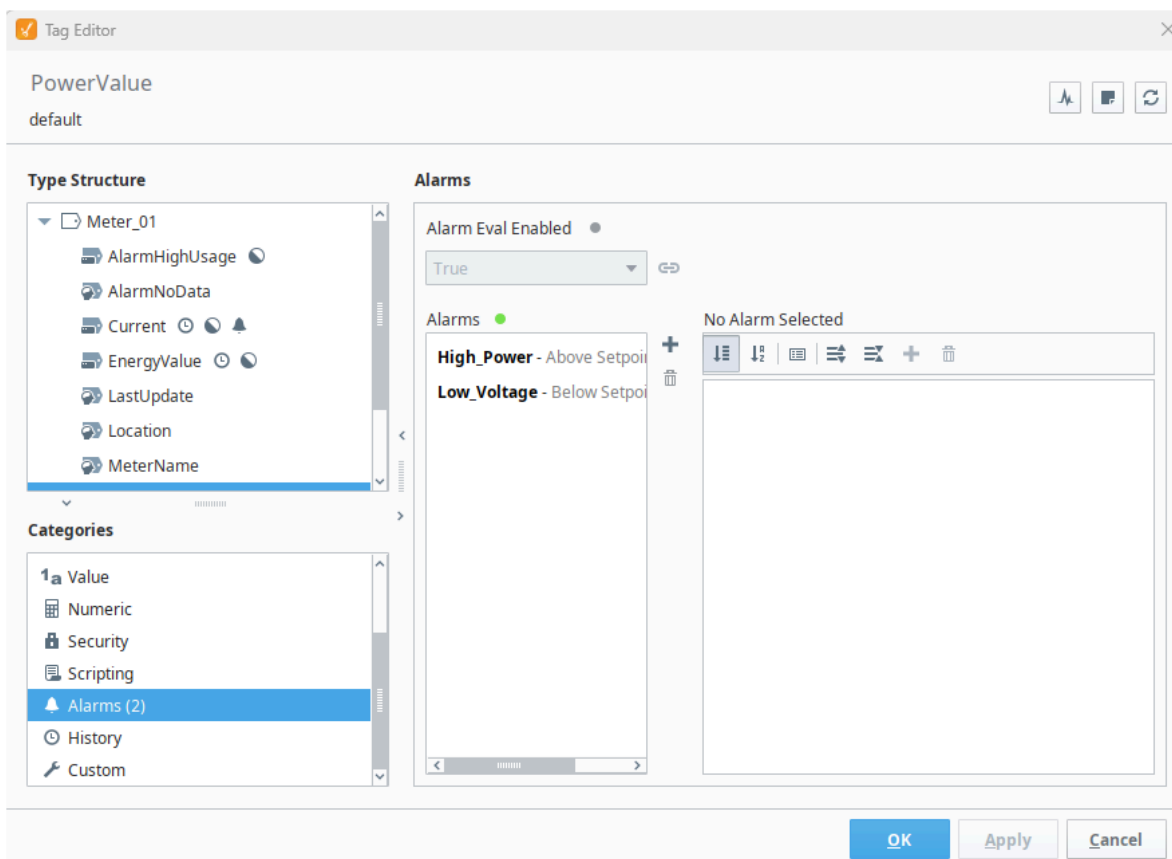
Tagi zostały zorganizowane w folderze EMS, z podziałem na moduł Energy oraz lokalizacje takie jak Production, Office, ServerRoom i Warehouse. Dla licznika Meter\_01 dostępne są wartości pomiarowe Current, EnergyValue, PowerValue i Voltage, a także tagi pomocnicze wykorzystywane przez alarmy i logikę dashboardu.

| Tag            | Value    |
|----------------|----------|
| EMS            |          |
| Alarm Metrics  |          |
| Calculations   |          |
| Energy         |          |
| Alarm Metrics  |          |
| Office         |          |
| Production     |          |
| Alarm Metrics  |          |
| Meter_01       |          |
| Alarm Metrics  |          |
| Parameters     |          |
| AlarmHighUsage | ✓        |
| AlarmNoData    | null     |
| Current        | 29       |
| EnergyValue    | 1 674,99 |
| LastUpdate     | null     |
| Location       | null     |
| MeterName      | null     |
| PowerValue     | 470,29   |
| Status         | ✓        |
| Voltage        | 230,35   |
| ServerRoom     |          |
| Warehouse      |          |
| System         |          |
| New Tag        | 79       |
| CurrentA1      | 29       |
| CurrentA2      | 79       |
| CurrentA3      | 124      |
| CurrentA4      | 52       |
| CurrentA5      | 69       |
| CurrentA6      | 138      |
| CurrentB1      | 196      |
| CurrentB2      | 103      |
| CurrentB3      | 159      |
| CurrentB4      | 135      |
| CurrentB5      | 126      |

Rys. 5. Struktura tagów EMS/Energy/Production/Meter\_01.

## 5.1. Alarmy na tagach

Alarmy zostały przypisane bezpośrednio do wartości pomiarowych licznika. Przykładowo tag PowerValue posiada alarm High\_Power oraz Low\_Voltage. Alarmy są powiązane z konkretnym licznikiem, dzięki czemu w Alarm Status Table można odczytać źródło alarmu, priorytet, czas aktywacji oraz stan potwierdzenia.



Rys. 6. Konfiguracja alarmów High\_Power i Low\_Voltage dla licznika Meter\_01.

## 6. Backend aplikacji - skrypty projektu

Logika aplikacji została wydzielona do modułów Project Library. Dzięki temu kod jest uporządkowany i może być wielokrotnie używany przez różne komponenty dashboardu. Rozwiązanie ogranicza duplikowanie logiki w komponentach Perspective.

### 6.1. Skrypt ems\_alarms

Moduł ems\_alarms odpowiada za zliczanie aktywnych alarmów. Funkcja getActiveAlarmCount() korzysta z system.alarm.queryStatus() i zwraca liczbę alarmów w stanach ActiveUnacked oraz ActiveAacked. Wynik jest prezentowany na karcie Active Alarms.

```

ems_alarms

1 def getActiveAlarmCount():
2     alarms = system.alarm.queryStatus(
3         state=["ActiveUnacked", "ActiveAcked"]
4     )
5     return len(alarms)

```

Rys. 7. Funkcja zliczająca aktywne alarmy.

## 6.2. Skrypt ems\_kpi

Moduł ems\_kpi pobiera wartości z tagów i przygotowuje dane dla kart KPI. Funkcje uwzględniają współczynniki lokalizacji oraz licznika, dzięki czemu wartości zmieniają się po użyciu filtrów w nagłówku dashboardu.

```

ems_kpi

1 def _read(path, default=0):
2     try:
3         qv = system.tag.readBlocking([path])[0]
4         if qv.value is None:
5             return default
6         return qv.value
7     except:
8         return default
9
10
11 def _to_float(value, default=1.0):
12     try:
13         return float(value)
14     except:
15         return default
16
17
18 def getCurrentPower(locationFactor=1.0, meterFactor=1.0):
19     current = float(_read("[default]EMS/Energy/Production/Meter_01/Current", 0))
20     locationFactor = _to_float(locationFactor, 1.0)
21     meterFactor = _to_float(meterFactor, 1.0)
22
23     return round(current * locationFactor * meterFactor, 2)
24
25
26 def getEnergyValue(locationFactor=1.0, meterFactor=1.0):
27     energy = float(_read("[default]EMS/Energy/Production/Meter_01/EnergyValue", 0))
28     locationFactor = _to_float(locationFactor, 1.0)
29     meterFactor = _to_float(meterFactor, 1.0)
30
31     return round(energy * locationFactor * meterFactor, 2)
32
33
34 def getVoltage(locationFactor=1.0, meterFactor=1.0):
35     locationFactor = _to_float(locationFactor, 1.0)
36     meterFactor = _to_float(meterFactor, 1.0)
37
38     voltage = 225.0 + (locationFactor * 8.0) + (meterFactor * 2.0)

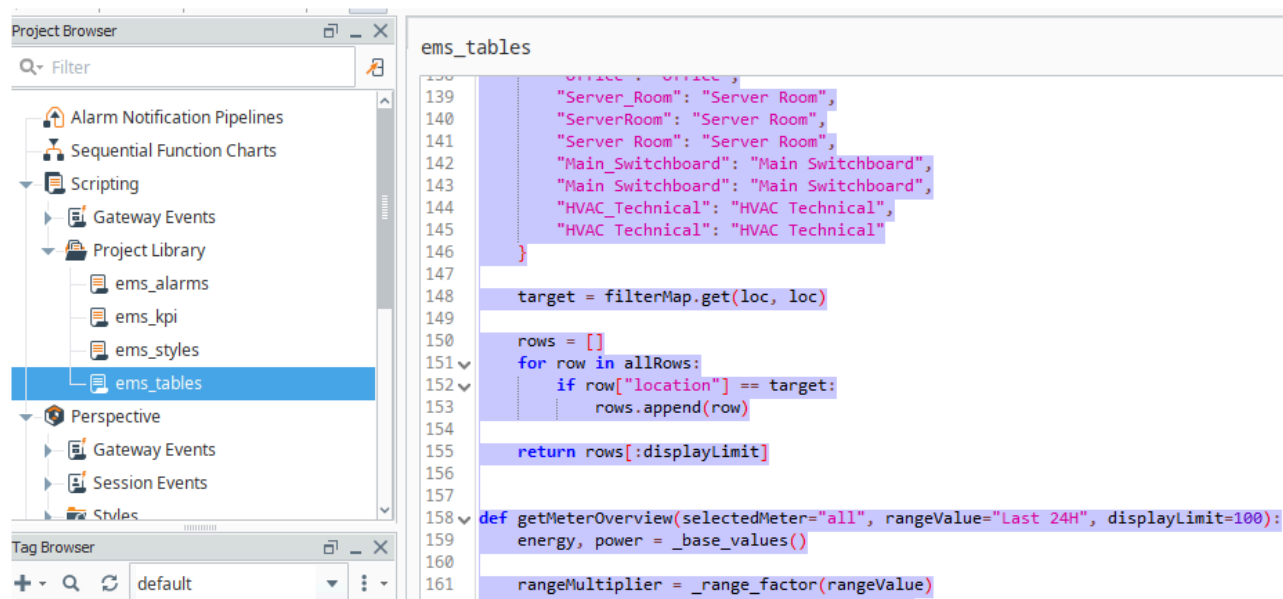
```

Rys. 8. Fragment skryptu ems\_kpi obsługującego wartości KPI.

### 6.3. Skrypt ems\_tables

Moduł ems\_tables generuje dane dla tabel: Energy trend, Consumption by location oraz Station / meter overview.

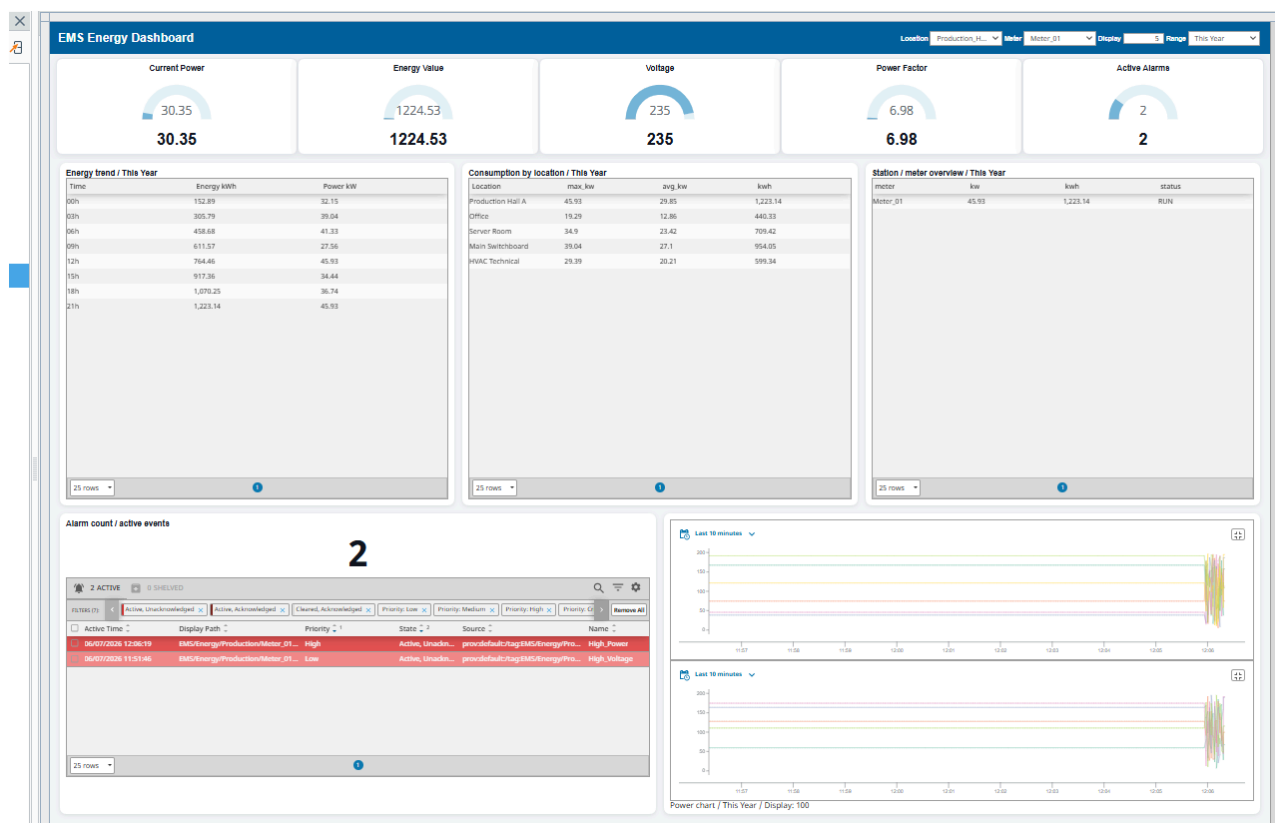
Skrypt obsługuje wybrany zakres czasu, limit wyświetlanych rekordów, lokalizację i licznik. Dzięki temu tabele reagują na filtry użytkownika.



Rys. 9. Fragment skryptu ems\_tables generującego dane dla tabel dashboardu.

## 7. Dashboard Perspective

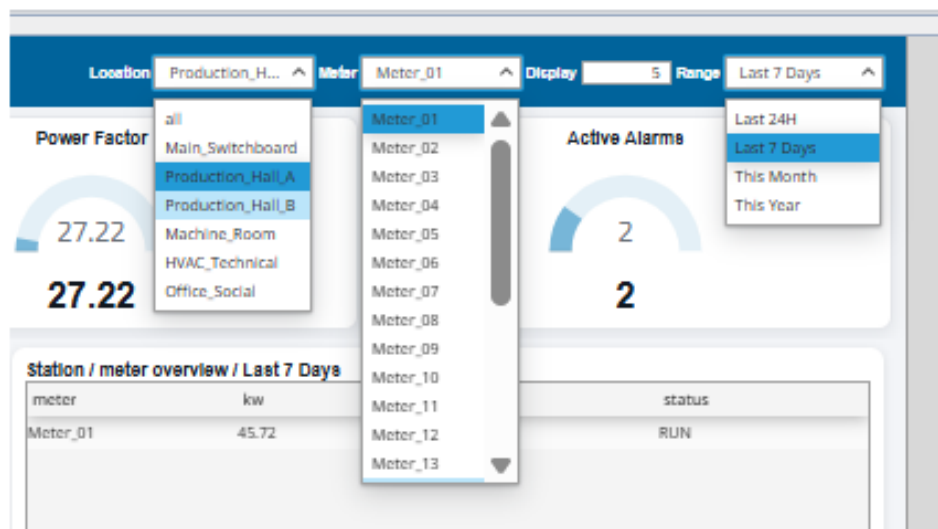
Główny widok Perspective pełni funkcję dashboardu operatorsko-analitycznego. Zawiera nagłówek z filtrami, karty KPI, tabele analityczne, komponent alarmowy oraz wykresy trendów czasowych. Interfejs został przygotowany jako jeden spójny panel monitorowania energii.



Rys. 10. Główny dashboard EMS Energy w module Perspective.

## 7.1. Filtry dashboardu

Nagłówek zawiera filtry Location, Meter, Display oraz Range. Filtry zapisują wybory użytkownika do właściwości view.custom, z których korzystają karty KPI i tabele. Dzięki temu komponenty znajdujące się w innych kontenerach reagują na wybór dokonany w Headerze.

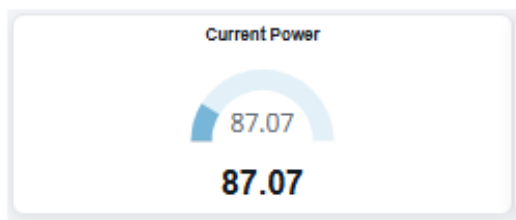


Rys. 11. Filtry Location, Meter, Display i Range.

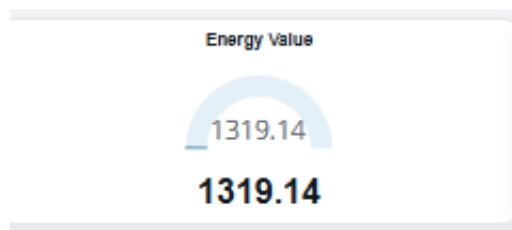


## 7.2. Karty KPI

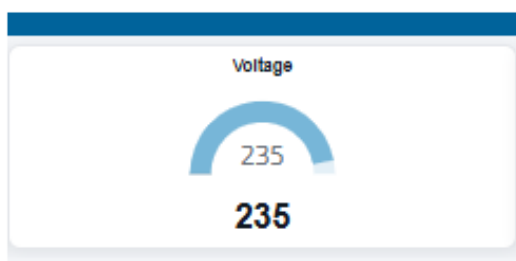
Karty KPI pokazują bieżące wartości najważniejszych parametrów systemu. Current Power prezentuje aktualną wartość obciążenia, Energy Value pokazuje zużycie energii, Voltage prezentuje napięcie, Power Factor przedstawia wartość pomocniczą/moc przeliczeniową, a Active Alarms pokazuje liczbę aktywnych alarmów.



Current Power

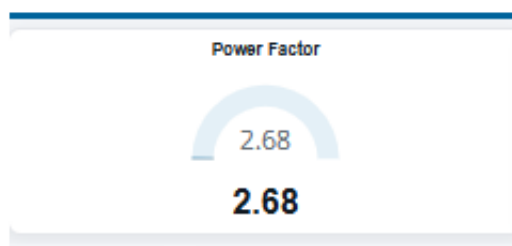


Energy Value

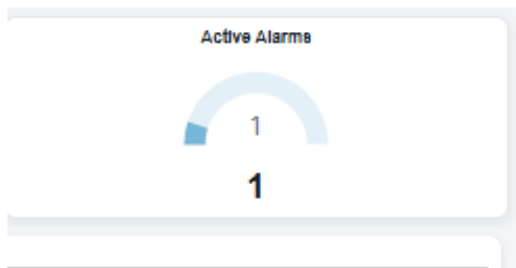


ication / Last 7 Days

Voltage



Power Factor



Active Alarms

Rys. 12. Przykładowe karty KPI systemu EMS Energy.

## 7.3. Tabele analityczne

Dashboard zawiera trzy główne tabele analityczne. Tabela Energy trend przedstawia zmianę energii i mocy w czasie. Consumption by location porównuje zużycie według lokalizacji. Station / meter overview pokazuje przegląd liczników, ich status oraz aktualne wartości kW i kWh.

**Energy trend / Last 7 Days**

| Time      | Energy kWh | Power kW |
|-----------|------------|----------|
| Day -6    | 1,376.18   | 28.8     |
| Day -5    | 2,752.36   | 34.97    |
| Day -4    | 4,128.54   | 37.03    |
| Day -3    | 5,504.72   | 24.69    |
| Day -2    | 6,880.9    | 41.14    |
| Yesterday | 8,257.08   | 30.86    |
| Today     | 9,633.26   | 32.91    |

25 rows

1

Rys. 13. Tabela Energy trend dla wybranego zakresu czasu.

**Consumption by location / Last 7 Days**

| Location          | max_kw | avg_kw | kwh      |
|-------------------|--------|--------|----------|
| Production Hall A | 42.95  | 27.92  | 9,730.55 |
| Office            | 18.04  | 12.03  | 3,503    |
| Server Room       | 32.64  | 21.9   | 5,643.72 |
| Main Switchboard  | 36.51  | 25.34  | 7,589.83 |
| HVAC Technical    | 27.49  | 18.9   | 4,767.97 |

25 rows

1

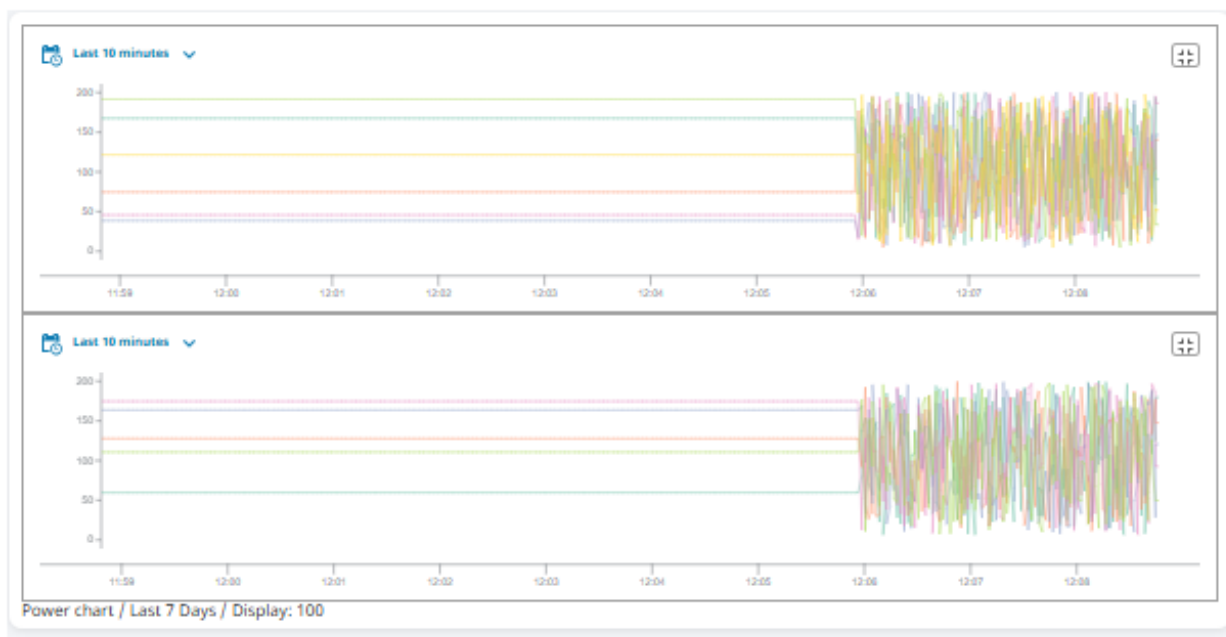
Rys. 14. Tabela Consumption by location.

| meter    | kw    | kwh      | status |
|----------|-------|----------|--------|
| Meter_01 | 19.55 | 9,779.19 | RUN    |

Rys. 15. Tabela Station / meter overview.

## 7.4. Wykresy czasowe

Sekcja wykresów prezentuje trendy czasowe danych pomiarowych. Wykresy umożliwiają obserwację zmian wartości i są opisane aktualnym zakresem czasu oraz limitem Display, dzięki czemu użytkownik widzi, jakiego przedziału dotyczy analiza.



Rys. 16. Wykresy trendów czasowych danych pomiarowych.

## 8. Alarmowanie

System alarmowania składa się z alarmów skonfigurowanych na tagach, licznika aktywnych alarmów oraz komponentu Alarm Status Table. Alarmy są prezentowane operatorowi z informacją o priorytecie, czasie aktywacji, Źródle, nazwie i stanie potwierdzenia.

Alarm count / active events

2

2 ACTIVE 0 SHELVED

FILTERS (7): Active, Unacknowledged x Active, Acknowledged x Cleared, Acknowledged x Priority: Low x Priority: Medium x Priority: High x Priority: Critical x Remove All

| Active Time         | Display Path                      | Priority | State             | Source                             | Name         |
|---------------------|-----------------------------------|----------|-------------------|------------------------------------|--------------|
| 06/07/2026 12:08:36 | EMS/Energy/Production/Meter_D1... | Medium   | Active, Unackn... | providefault/tag:EMS/Energy/Pro... | Low_Voltage  |
| 06/07/2026 11:51:46 | EMS/Energy/Production/Meter_D1... | Low      | Active, Unackn... | providefault/tag:EMS/Energy/Pro... | High_Voltage |

25 rows

Rys. 17. Alarm Status Table z aktywnymi alarmami.

Licznik alarmów jest pobierany dynamicznie przez skrypt `ems_alarms`. Tabela alarmowa pokazuje realne zdarzenia systemu Ignition, dzięki czemu operator ma szybki dostęp do aktualnych problemów w instalacji.

## 9. Bezpieczeństwo i role

W bazie danych przygotowano tabele `app_users`, `roles` oraz `user_roles`. Umożliwiają one logiczne odwzorowanie użytkowników i ról systemowych. Struktura ta może być wykorzystana do implementacji poziomów bezpieczeństwa w Ignition, zgodnie z rolami Administrator, Engineer, Operator i Analyst.

| Rola          | Zakres uprawnień w projekcie                                   |
|---------------|----------------------------------------------------------------|
| Administrator | pełna konfiguracja systemu i dostęp administracyjny            |
| Engineer      | dostęp do dashboardów, diagnostyki i konfiguracji technicznej  |
| Operator      | bieżący podgląd statusu, alarmów i podstawowych ekranów        |
| Analyst       | analiza danych, tabel i wykresów bez ingerencji w konfigurację |

## 10. Testowanie działania

Projekt został przetestowany w zakresie działania połączenia z bazą, symulatora, historiana, tagów, alarmów oraz komponentów dashboardu. Szczególną uwagę zwrócono na spójność licznika alarmów z Alarm Status Table oraz reakcję KPI i tabel na filtry użytkownika.

- połączenie EMS\_PostgreSQL ma status VALID;
- symulator enea ma status RUNNING;
- SQL Historian historia ma status RUNNING;
- tagi Meter\_01 zwracają wartości Current i EnergyValue;
- licznik Active Alarms zmienia się zgodnie z aktywnymi alarmami;
- filtry Location, Meter, Range i Display wpływają na prezentowane dane;
- tabele analityczne generują dane przez skrypty Project Library.

## 11. Podsumowanie

Przygotowany system EMS Energy realizuje podstawowe wymagania projektu: posiada połączenie z bazą PostgreSQL, działający symulator urządzenia, historian, strukturę tagów, alarmy, skrypty backendowe oraz dashboard Perspective. Interfejs umożliwia monitorowanie najważniejszych wskaźników energetycznych, analizę danych w tabelach i wykresach oraz bieżącą obsługę alarmów.

Projekt jest skalowalny, ponieważ struktura lokalizacji, liczników, tagów i tabel pozwala na dodawanie kolejnych urządzeń pomiarowych. Wydzielenie logiki do Project Library ułatwia utrzymanie i dalszą rozbudowę systemu.